

Index Attribute in Entity Framework Core

In this article, I will discuss the **Index Data Annotation Attribute in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [ForeignKey Attribute in Entity Framework Core](#) with Examples. At the end of this article, you will understand how to create Indexes (Clustered, Non-Clustered, and Unique or Non-Unique Indexes) using EF Core with Examples.

What is an Index in a Database?

An index in a database significantly improves the speed of data retrieval operations. It allows the database to quickly locate rows without scanning the entire table. An index works similarly to the index at the back of a book: instead of searching through every page, you can use the index to directly find the relevant content.

When we create an index on a table column or columns, the database stores a sorted version of the data in those columns, along with pointers to the actual data rows. This allows for efficient data lookups rather than performing a full table scan. Using indexes leads to faster queries but costs additional storage space and maintenance during insert, update, and delete operations.

Why Do We Need an Index?

Indexes are crucial for improving the performance of database read operations in the following scenarios:

- **Faster Query Performance:** Indexes significantly speed up data retrieval for frequently executed queries, especially in large tables. Without an index, the database may need to perform a full table scan, which is slower.
- **Efficient Sorting:** Indexes help the database efficiently sort data based on indexed columns, which is useful for queries with ORDER BY clauses.
- **Quick Search for Specific Values:** Indexes improve the efficiency of searching for specific values or ranges, such as with WHERE conditions.
- **Improved JOIN Performance:** Indexes on columns used in JOIN operations enhance the efficiency of matching rows between tables.

Default Indexes in Entity Framework Core (EF Core)

When generating a database schema using EF Core, certain indexes are created automatically:

- **Primary Key Index:** EF Core automatically creates a clustered index on the primary key column. A clustered index determines the physical order of data in a table, meaning that rows are stored in the order of the primary key. Since the primary key uniquely identifies each record, indexing it enhances retrieval performance.
- **Foreign Key Index:** EF Core automatically creates a non-clustered index on foreign key columns. These indexes improve the performance of JOIN operations, which are commonly used to link related tables.

Custom Indexes in EF Core

Now, if you want to create Indexes on any other columns apart from the Primary Key and foreign key columns, you need to configure them using Index Attribute or Fluent API. Let us proceed and understand how to use Index Attribute in this article. In our upcoming sessions, we will discuss how to use Fluent API configuration.

Index Attribute in Entity Framework Core:

The [Index] attribute in Entity Framework Core allows developers to define indexes directly on entity class. When migrations are applied, EF Core creates these indexes in the database. These indexes help the database perform efficient search, sort, and filter operations on the indexed columns.

If you go to the definition of Index Attribute, you will see the following. As you can see, it is a sealed class with two constructors and a few properties.

```
namespace Microsoft.EntityFrameworkCore
{
    public sealed class IndexAttribute : Attribute
    {
        private string _name;

        private bool? _isUnique;

        private bool[] _isDescending;

        private bool _allDescending;

        public IReadOnlyList<string> PropertyNames
        {
            get { return _propertyNames; }
        }

        public string? Name
        {
            get { return _name; }
        }

        public bool IsUnique
        {
            get { return _isUnique == true; }
        }

        public bool[]? IsDescending
        {
            get { return _isDescending; }
        }

        public bool AllDescending
        {
            get { return _allDescending; }
        }

        public bool IsUniqueHasValue => _isUnique.HasValue;

        public IndexAttribute(string propertyName, params string[] additionalPropertyNames)
        {
            _name = propertyName;
            _propertyNames = new string[] { propertyName };
        }

        public IndexAttribute(params string[] propertyNames)
        {
            _name = string.Join("_", propertyNames);
            _propertyNames = propertyNames;
        }
    }
}
```

Key Properties of the Index Attribute

- **IsUnique:** Specifies whether the index enforces uniqueness across the indexed columns.
- **Name:** Allows us to specify a custom name for the index. If not provided, EF Core generates a name in the format **IX_{Entity}_{Property}**.
- **IsDescending / AllDescending:** Configures the sort order for the indexed columns. **IsDescending** specifies the order for individual columns, while **AllDescending** sets all columns to descending order.

How to Apply the Index Attribute in EF Core:

The [Index] attribute is applied at the class level. You specify the properties to be indexed in the constructor of the attribute. Let us modify the Student entity class as follows to use the Index Attribute. Here, you can see we have applied the Index Attribute on the Student class, and to the constructor, we are passing the RegistrationNumber property. This will create a non-unique, non-clustered index on the RegistrationNumber column with the default naming convention.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [Index(nameof(RegistrationNumber))] // Index on the RegistrationNumber
    column
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
        public int RegistrationNumber { get; set; }
    }
}
```

```
}
```

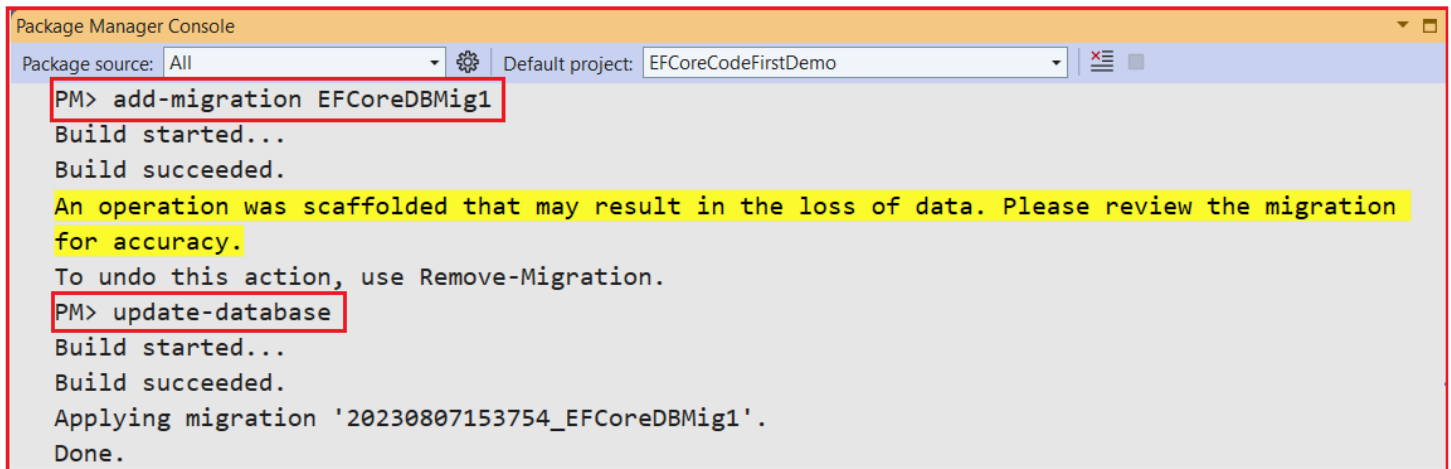
Modifying the Context Class:

Next, modify the EFCoreDbContext class as follows:

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        public DbSet<Student> Students { get; set; }
    }
}
```

Creating Migration and Syncing with Database

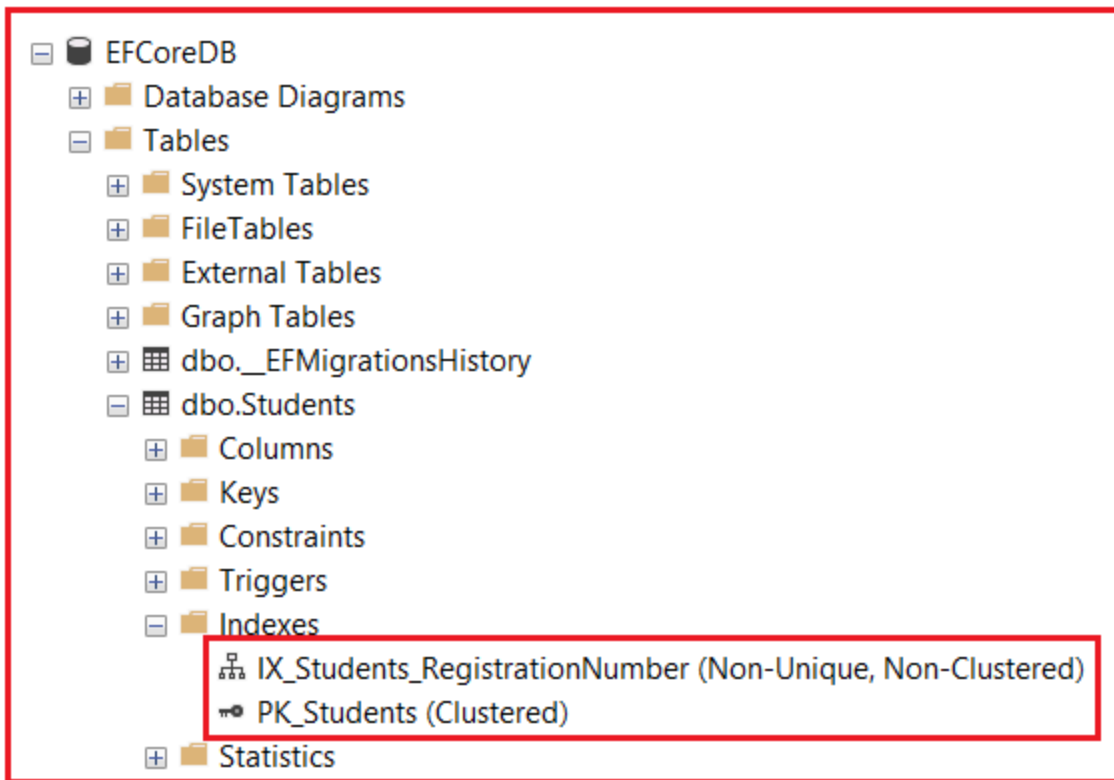
Open Package Manager Console and Execute **add-migration** and **update-database** commands as shown in the below image:

The screenshot shows the Package Manager Console window for the project 'EFCoreCodeFirstDemo'. The 'Package source' is set to 'All'. The command 'PM> add-migration EFCoreDBMig1' is entered and executed. The output shows 'Build started...' and 'Build succeeded.' followed by a warning: 'An operation was scaffolded that may result in the loss of data. Please review the migration for accuracy.' and 'To undo this action, use Remove-Migration.' Then, the command 'PM> update-database' is entered and executed. The output shows 'Build started...', 'Build succeeded.', 'Applying migration '20230807153754_EFCoreDBMig1'', and 'Done.'

```
Package Manager Console
Package source: All Default project: EFCoreCodeFirstDemo
PM> add-migration EFCoreDBMig1
Build started...
Build succeeded.
An operation was scaffolded that may result in the loss of data. Please review the migration for accuracy.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230807153754_EFCoreDBMig1'.
Done.
```

Verifying the Index in the Database:

By default, it will create the Index with the name **IX_{Table name}_{Property Name}**. As we have Applied the Index Attribute on the **RegistrationNumber** property, EF Core will create the Index with the name **IX_Students_RegistrationNumber**, as shown in the image below. As you can see, by default, the Index is created as **Non-Unique** and **Non-Clustered**. Later, I will show you how to create a unique Index.



Now, you might have one question: We applied the Index Attribute to a Single Property, but here, we can see two indexes. How is that possible? Yes, it is possible. This is because, by default, one Clustered Index is created when we create the Primary Key in a database.

Creating a Custom-Named Index in EF Core

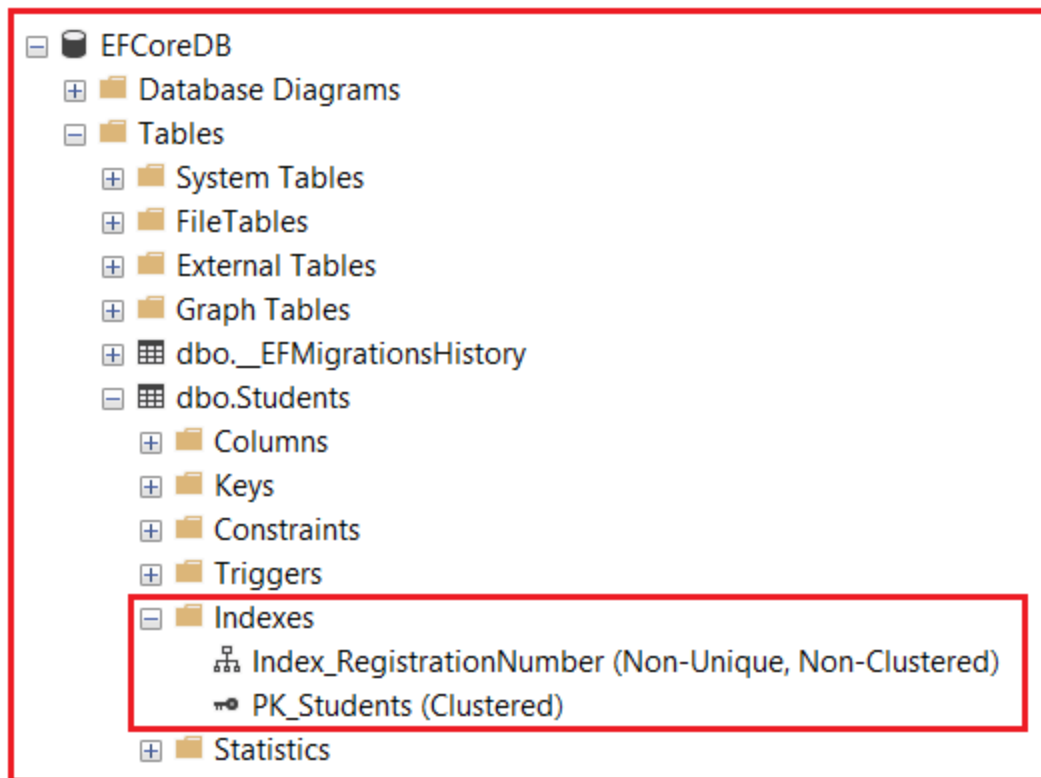
Now, if you want to give a different name to the Index name rather than the auto-generated index name, you need to use the other overloaded version of the constructor, which takes the Name parameter. For a better understanding, modify the Student entity as follows. Here, you can see we are providing the name as **Index_RegistrationNumber**.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [Index(nameof(RegistrationNumber), Name = "Index_RegistrationNumber")]
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
        public int RegistrationNumber { get; set; }
    }
}
```

Open Package Manager Console and Execute **add-migration** and **update-database** commands as follows.

```
Package Manager Console
Package source: All [v] [gear] Default project: EFCoreCodeFirstDemo [v]
PM> add-migration EFCoreDBMig2
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230807154614_EFCoreDBMig2'.
Done.
```

Now, it should create the index with the specified name in the database, as shown in the image below.



Composite Index (Multiple Columns) in EF Core:

We can also create an index on multiple columns to improve the performance of queries involving both columns. For this, we need to specify property names separated by a comma. Let us understand this with an example.

Please modify the Student entity as follows. As you can see here, we have specified the RegistrationNumber and RollNumber properties to the constructor of the Index Attribute. The Entity Framework Core will create one composite Index based on the RegistrationNumber and RollNumber columns with the custom name Index_RegistrationNumber_RollNumber.

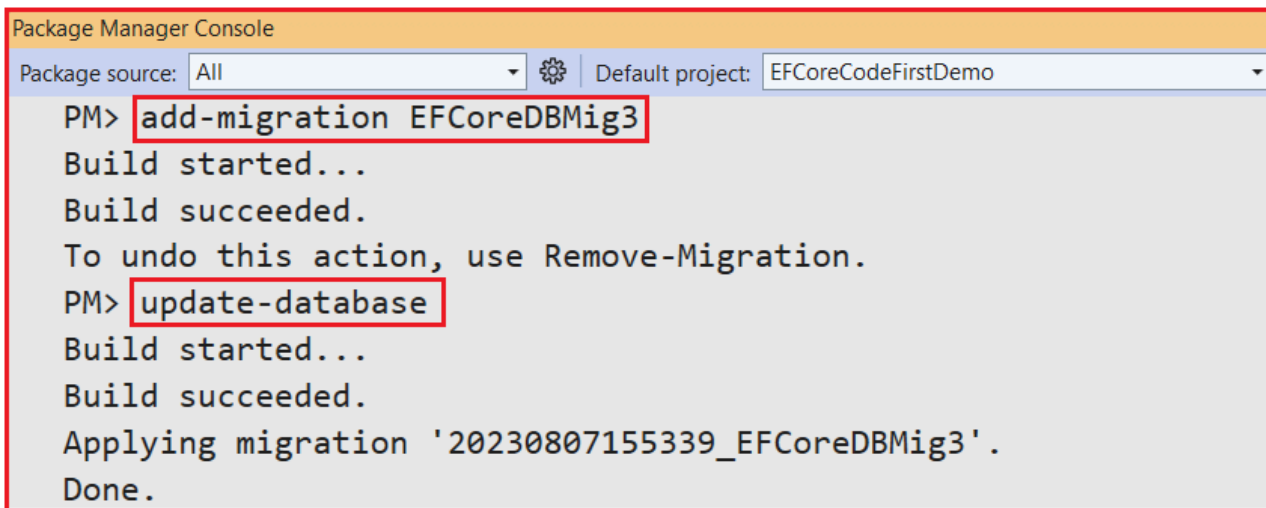
```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
```

```

[Index(nameof(RegistrationNumber), nameof(RollNumber), Name =
"Index_RegistrationNumber_RollNumber")]
public class Student
{
    public int StudentId { get; set; }
    public string? FirstName { get; set; }
    public string? LastName { get; set; }
    public int RegistrationNumber { get; set; }
    public int RollNumber { get; set; }
}
}

```

Now, again, open Package Manager Console and Execute the following **add-migration** and **update-database** commands.



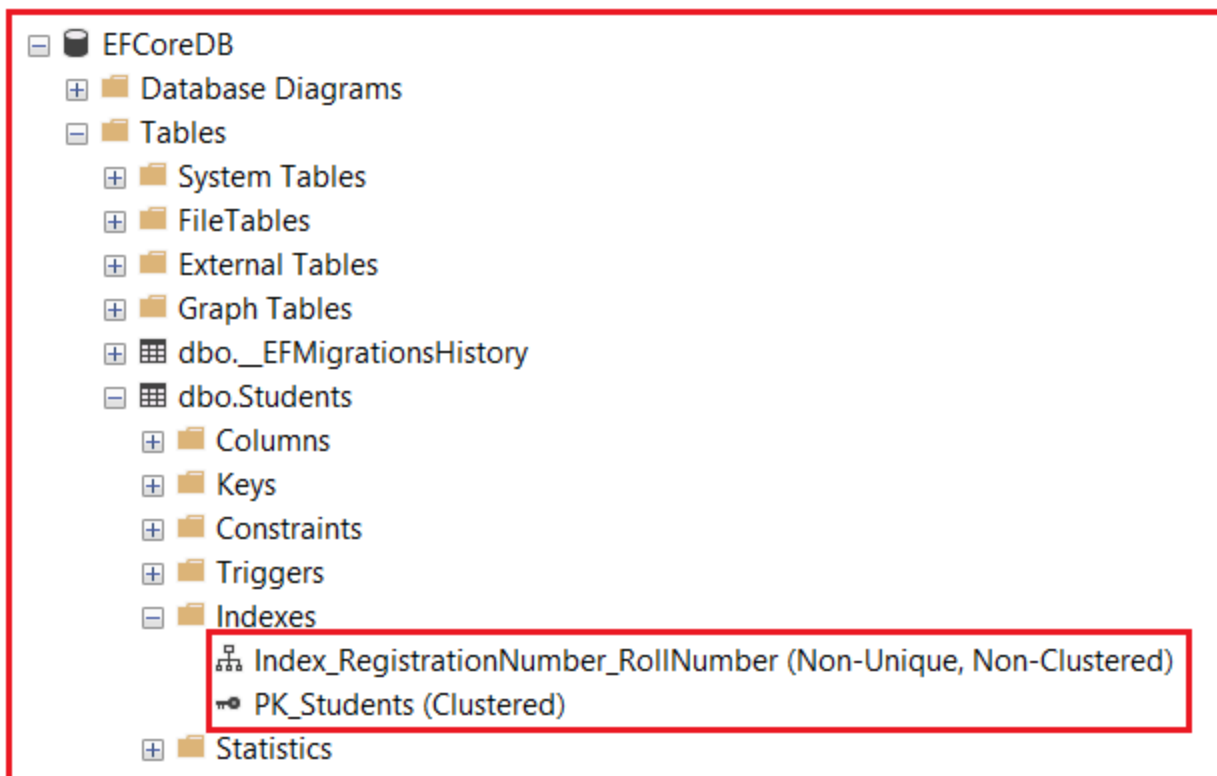
The screenshot shows the Package Manager Console interface. At the top, there are two dropdown menus: 'Package source: All' and 'Default project: EFCoreCodeFirstDemo'. Below these, the console output shows the following sequence of commands and messages:

```

PM> add-migration EFCoreDBMig3
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230807155339_EFCoreDBMig3'.
Done.

```

Now, if you verify the database, it should create the index with the specified name based on the two columns shown in the image below.



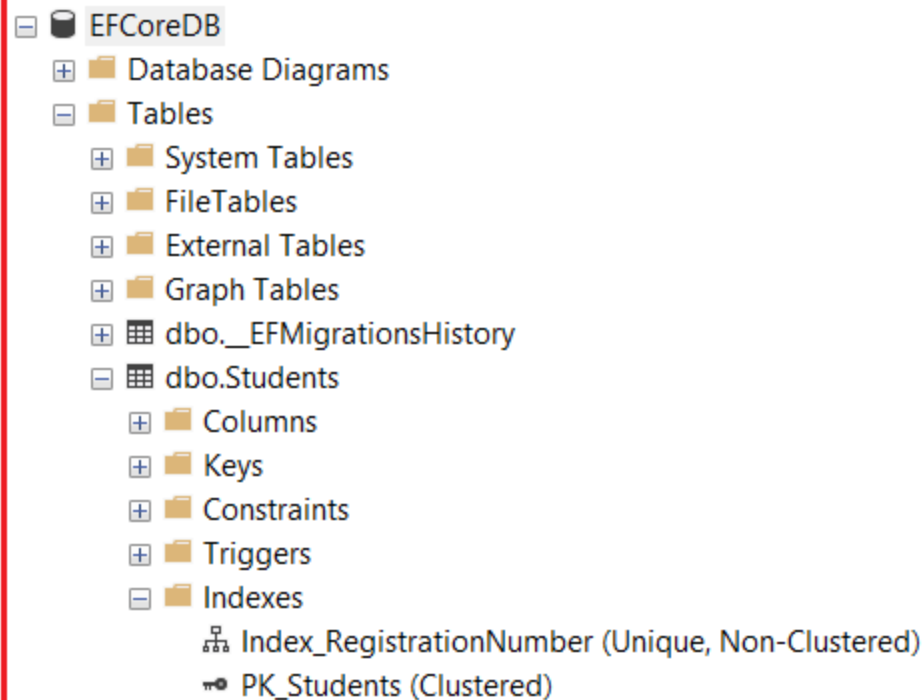
How Do We Create Clustered and Unique Indexes Using Entity Framework Core?

By default, Entity Framework Core creates a Non-Clustered and Non-Unique Index. To create a Unique Index, we need to use the **IsUnique** property and set its values to True. **It is impossible to manually create the Clustered Index using the Index Attribute in EF Core.** A table can have a maximum of 1 clustered index, which will be created on the primary key column by default, and we cannot change this default behavior.

To better understand, please modify the Student entity class as follows to create a Unique Non-Clustered Index on the RegistrationNumber property.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [Index(nameof(RegistrationNumber), Name = "Index_RegistrationNumber",
    IsUnique = true)]
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
        public int RegistrationNumber { get; set; }
    }
}
```

With the above changes, open the Package Manager Console and Execute the **add-migration** and **update-database** commands. Then, verify the database; it should create a unique index with the specified name, as shown in the image below.



How to Specify the Index Sort Order in EF Core:

In most databases, each column covered by an index can be either ascending or descending. For indexes covering only one column, this typically does not matter. However, the ordering is important for composite indexes to perform well.

By default, an index's sort order is ascending. However, we can control the sort order for each column in a composite index. We can arrange all columns in descending order as follows. Here, we are using the `AllDescending` property and setting its value to `true`, which will arrange all columns in descending order.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [Index(nameof(RegistrationNumber), nameof(RollNumber), AllDescending =
true, Name = "Index_RegistrationNumber_RollNumber")]
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
        public int RegistrationNumber { get; set; }
        public int RollNumber { get; set; }
    }
}
```

Specifying Different Order for Different Columns:

We can also specify the sort order on a column-by-column basis. For a better understanding, please modify the `Student` class as follows. Here, we use the `IsDescending` property, passing one anonymous array and specifying the value as `false` or `true`. In this case, the index on `RegistrationNumber` will be descending, the index on `RollNumber` will be ascending, and the index name will be `Index_RegistrationNumber_RollNumber`.

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [Index(nameof(RegistrationNumber), nameof(RollNumber), IsDescending =
new[] { false, true }, Name = "Index_RegistrationNumber_RollNumber")]
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
        public int RegistrationNumber { get; set; }
        public int RollNumber { get; set; }
    }
}

```

Can we Create Multiple Indexes in a Table using EF Core?

Yes. Using EF Core, it is possible to create multiple indexes on a table. To achieve this, we must decorate the Entity with Multiple Index Attributes.

For a better understanding, please modify the Student Entity as follows. Here, you can see we are creating two composite indexes. One index on the FirstName and LastName column with the name Index_FirstName_LastName. The other index is on the RegistrationNumber and RollNumber columns with the name Index_RegistrationNumber_RollNumber.

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [Index(nameof(FirstName), nameof(LastName), Name =
"Index_FirstName_LastName")]
    [Index(nameof(RegistrationNumber), nameof(RollNumber), Name =
"Index_RegistrationNumber_RollNumber")]
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
        public int RegistrationNumber { get; set; }
        public int RollNumber { get; set; }
    }
}

```

When Should We Use Index Attribute in EF Core:

The Index Attribute should be used in the following scenarios to improve query performance:

- **Frequent Search or Filtering Operations:** When columns are frequently queried (e.g., WHERE, GROUP BY). Create an Index on columns used in WHERE and GROUP BY clauses.
- **Efficient Sorting:** When sorting data on specific columns (e.g., ORDER BY). Add an index on columns frequently used in ORDER BY.
- **Speeding Up JOIN Operations:** When columns are used in JOIN operations, particularly foreign keys. Create an Index on foreign keys for efficient JOIN operations.

- **Preventing Duplicate Entries:** Use unique indexes for email addresses or usernames to enforce uniqueness.
- **Complex Queries (Composite Indexes):** When queries involve multiple columns together. Use composite indexes for queries involving multiple columns.

When Not to Use Indexes:

- **Write-Heavy Tables:** Indexes can slow down insert, update, and delete operations because the index must be updated each time the data changes.
- **Small Tables:** Indexing may not provide a significant performance boost for small tables with only a few rows, and a full table scan may be faster.
- **Frequent Updates on Indexed Columns:** Index maintenance can be costly if the indexed columns are updated frequently.

Indexes optimize database performance in Entity Framework Core. We can significantly enhance query efficiency by applying the [Index] attribute to entity classes, especially for large and complex databases. However, it's essential to balance the benefits of faster reads with the overhead of additional storage and potentially slower in write-heavy scenarios.

In the next article, I will discuss [InverseProperty Attribute in Entity Core](#) with Examples. I explain Index Data Annotation Attribute in Entity Framework Core with Examples in this article. I hope you enjoyed this Index Attribute in EF Core with Examples article. Please provide your valuable feedback and suggestions for this article.